Mikhail Razakov

Compiler / LLVM Engineer · Optimization & Analysis

misha13022008@gmail.com
t.me/Micodiy
github.com/Misha1302

Compiler / LLVM engineer with professional compiler and static-analysis work. At MCST, I implemented LLVM 22 loop/interprocedural analyses and conservative legality checks for IR transformations; at ISP RAS, I contribute to SharpChecker. Independent projects add fixed-point Roslyn CFG analysis, x86-64 codegen/register allocation, and UniversalToolchain compiler infrastructure.

EXPERIENCE

2026 — present — **ISP RAS — Static Analysis Engineer**

- Contribute to SharpChecker, ISP RAS's industrial static-analysis platform for C#/.NET.

July — August 2026 — **MCST — Compiler Engineering Intern**

- Implemented an LLVM LICM pass with loop analysis, side-effect/speculative-safety checks, and hoisting of loop-invariant instructions to preheaders.
- Developed a conservative interprocedural global-IV prototype with APInt affine evolution, transitive call effects, a separate legality phase, and LLVM IR transformation; unsupported CFG/call cases are rejected.

PROJECTS

LLVM Interprocedural Global IV Optimization

An LLVM 22 pass for interprocedural analysis of linear global-variable evolution and safe IR transformation, using APInt, loop/dominator analysis, transitive effects, and must-execute conditions.

29 positive and negative regression tests, LLVM Verifier checks, idempotence checks, and before/after behavioral comparison.

UniversalToolchain

Typed artifact contracts; dependency/conflict and capability/provider resolution; deterministic pass ordering and artifact routes compile into an immutable LanguagePlan before execution.

Runtime validates the exact planned package/component graph and materializes selected components without re-planning; determinism/exact-binding checks, ownership guards, and interpreter/CIL parity protect the contract.

DerefAfterNullAnalyzer

Roslyn ControlFlowGraph with forward fixed-point data-flow: Unknown/MaybeNull/NotNull states propagate across basic blocks and merge at join points.

Successors are reprocessed until the state stabilizes, producing diagnostics for potentially unsafe dereference paths.

x86-64 Codegen & Register Allocation Lab

SSA-like IR, CFG/SSA validation, liveness/interference, register allocation with an independent assignment verifier, phi lowering, and SysV x86-64 codegen.

A reference interpreter and differential execution check generated-code semantics; the allocator contract is verified independently.

SKILLS

Compilers

C++23, LLVM 22, LLVM IR, optimization passes, interprocedural analysis, CFG/SSA, dominators, loop analysis, LICM, legality analysis

Static / Program Analysis

C#, .NET, Roslyn ControlFlowGraph, fixed-point data-flow analysis, state propagation and joins

Codegen

Rust, SysV x86-64, liveness/interference, register allocation, phi lowering, differential execution

Compiler Infrastructure

typed artifact contracts, deterministic planning/pass ordering, artifact routing, backend/runtime composition

EDUCATION

HSE University — Software Engineering, 2026–2030

RECOGNITION

LangDev'26 — Build the Language, Then Make the Abstractions Disappear

Accepted talk on UniversalToolchain's architecture; LangDev'26 takes place October 8–9, 2026 in Málaga.

UniversalToolchain — Baltic Science and Engineering Competition

First Degree Diploma and Grand Prize "Perfection as Hope" for UniversalToolchain.

MEPhI Junior

Overall winner; First Degree Diploma, 96/100 for UniversalToolchain.

Moscow / remote